

Probing the Symbolic Logical Reasoning Ability of Large Language Models

JIANCHAO JI, ZELONG LI, SHUYUAN XU, WENYUE HUA, JUNTAO TAN, HAOMING GONG, and YONGFENG ZHANG, Department of Computer Science, Rutgers University, New Brunswick, New Jersey, USA

Large Language Models (LLMs) have achieved significant successes in various research domains by learning the relationship between words. However, while these models are capable of making predictions and inferences based on the learned patterns, they lack logical reasoning abilities, which are crucial for solving problems in both theoretical and practical domains. In addition, traditional logic inference methods are effective in solving problems that are based on logic, but not suitable for general tasks such as recommendations.

In response to these challenges, this article introduces a Logical Large Language Model (L3M) that integrates the strengths of logical reasoning and LLMs. The data in L3M are represented in logical expressions, and the model uses logical constraints to learn the rules of basic logical operations such as And, Or, and Not. We conduct experiments on both theoretical tasks (solving logical equations) and practical tasks (recommender systems). The results of our theoretical experiments demonstrate that L3M is highly effective in solving logical expressions and variables. Additionally, L3M outperforms the state-of-the-art recommendation models in sequential recommendation tasks.

CCS Concepts: • **Information systems** → **Information retrieval**;

Additional Key Words and Phrases: Large Language Models, Information Retrieval, Recommendation Systems, Logical Reasoning

ACM Reference format:

Jianchao Ji, Zelong Li, Shuyuan Xu, Wenyue Hua, Juntao Tan, Haoming Gong, and Yongfeng Zhang. 2026. Probing the Symbolic Logical Reasoning Ability of Large Language Models. *ACM Trans. Intell. Syst. Technol.* 17, 4, Article 75 (April 2026), 22 pages.
<https://doi.org/10.1145/3729238>

This work was supported in part by National Science Foundation (NSF) IIS-2046457. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect those of the sponsors. Authors' Contact Information: Jianchao Ji (corresponding author), Department of Computer Science, Rutgers University, New Brunswick, New Jersey, USA; e-mail: jianchao.ji@rutgers.edu; Zelong Li, Department of Computer Science, Rutgers University, New Brunswick, New Jersey, USA; e-mail: zl359@rutgers.edu; Shuyuan Xu, Department of Computer Science, Rutgers University, New Brunswick, New Jersey, USA; e-mail: shuyuan.xu@rutgers.edu; Wenyue Hua, Department of Computer Science, Rutgers University, New Brunswick, New Jersey, USA; e-mail: wenyue.hua@rutgers.edu; Juntao Tan, Department of Computer Science, Rutgers University, New Brunswick, New Jersey, USA; e-mail: juntao.tan@rutgers.edu; Haoming Gong, Department of Computer Science, Rutgers University, New Brunswick, New Jersey, USA; e-mail: hg343@scarletmail.rutgers.edu; Yongfeng Zhang, Department of Computer Science, Rutgers University, New Brunswick, New Jersey, USA; e-mail: yongfeng.zhang@rutgers.edu.



This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).

ACM 2157-6912/2026/4-ART75

<https://doi.org/10.1145/3729238>

1 Introduction

The **Large Language Model (LLM)** has received significant attention in recent years, owing to its success in a variety of research areas, including **Natural Language Processing (NLP)** [4, 6, 66], computer vision [13, 30, 42], and recommender systems [17]. As an example, GPT-3 [4] has been utilized to generate human-like text for chatbots and automated customer service, while DALL-E [42] has been employed to generate high-quality images from text descriptions.

On the one hand, improving the architecture of LLMs has been shown to offer several advantages, such as improved results and more efficient data utilization [11, 48]. The Transformer architecture employed by TransRec [57], for instance, enables effective handling of sequential data, allowing TransRec to model the dependencies between different items in a user's interaction history and produce more accurate recommendations. P5 [17] converts data, such as user-item interactions, user descriptions, and reviews, into natural language sequences, unifying various recommendation tasks in a shared framework and achieving remarkable performance on all tasks.

On the other hand, although LLMs exhibit good generalization capabilities, most current LLMs still lack good logical reasoning ability [65]. Logical reasoning is a crucial component, especially in challenging problems such as symbolic reasoning [46], arithmetic [20], question answering [62], and recommender systems [5]. Specifically, tasks such as symbolic reasoning necessitate the ability to manipulate abstract variables and deduce logical conclusions. Recommendation systems, similarly, require the ability to comprehend underlying user preferences and make informed recommendations based on that understanding. The absence of logical reasoning capability in current LLMs can lead to suboptimal performance on these tasks and hinder their practical application in real-world scenarios.

The integration of reasoning into LLMs has seen some progress in solving various tasks such as arithmetic problems [8, 16, 20, 37, 54, 55, 69], text generation [7, 12, 53], and problem decomposition [14, 15, 28]. However, the existing methods have some limitations, such as the inability to handle complex reasoning scenarios. For instance, the BIG-Bench method [47] attempts to improve the accuracy of logical equation solving by increasing the training sequence number, but it is limited to simple reasoning equations. To address the need for logical reasoning in both theoretical and practical problems, **Neural Logical Reasoning (NLR)** [46] was proposed as a novel approach that integrates logical regularizers into neural networks. NLR trains basic logic operators such as And, Or, and Not as neural modules, and these modules are then used to solve logical equation problems. Furthermore, NLR has the ability to uncover the logical relations between users and items in e-commerce scenarios and has shown potential in improving sequential recommendation problems. On the other hand, **Neural Collaborative Reasoning (NCR)** [5] changed the encoding method of users' purchase history and achieved improved performance on sequential recommendation tasks compared to NLR.

In this work, we address the crucial importance of both logical reasoning and LLMs and investigate the integration of the two in recommendation systems. To achieve this, we propose the **Logical Large Language Model (L3M)**, a language model based on logical prompts that integrates the generalization ability of LLMs and logical reasoning. The L3M framework consists of an encoder and a decoder, where the input is processed by a bidirectional text encoder and the output is generated by a text decoder that operates in an autoregressive manner.

A key challenge in logical reasoning is learning the structure of the logical expression, which is composed of a set of variables and can differ significantly from other expressions with the same variables. To tackle this challenge, we convert the logical expression into a natural language format so that it can be learned by the L3M model. Additionally, we design logical prompts for the logical expression to ensure that the model learns the rules of logical operators correctly. Taking Figure 1

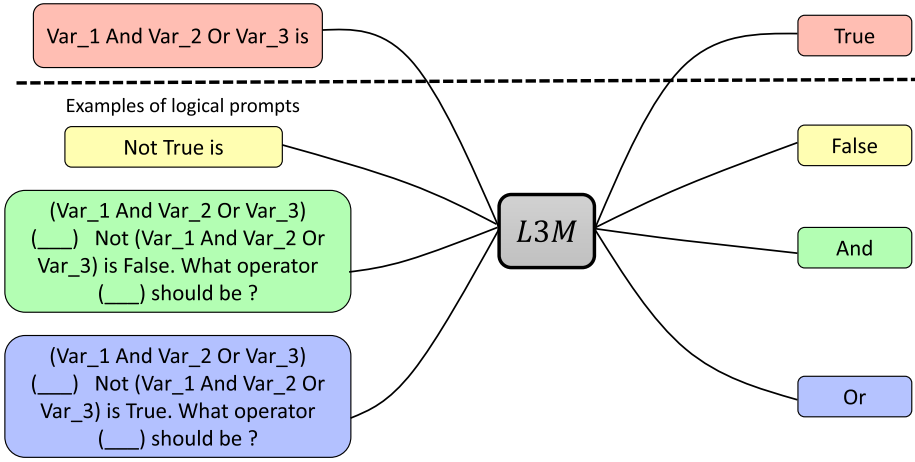


Fig. 1. An illustration of L3M. We design logical prompts to constrain the learning of L3M.

as an example, the original logical expression is:

$$v_1 \wedge v_2 \vee v_3 = \text{True}. \quad (1)$$

The first step in the implementation of L3M involves converting the original logical expression into a natural language format. For example, the expression $v_1 \wedge v_2 \vee v_3 = \text{True}$ is converted to “Var_1 And Var_2 Or Var_3 is True,” where “Var_1 And Var_2 Or Var_3” serves as the input and “True” is the expected label to be predicted by the model. To enhance the learning process of the logical operators, logical prompts are applied to the logical expression. Both the logical prompts and the converted logical expression are fed into the model during the training phase.

Empirical evaluations on theoretical problems, such as solving logical equations, indicate that L3M outperforms traditional LLMs in terms of logical inference ability. Furthermore, extensive experiments conducted on recommendation tasks demonstrate that incorporating logic constraints in the natural language input leads to significantly improved performance of L3M.

The key contributions of this article can be summarized as follows:

- We introduce a novel approach, L3M, to enhance the logical reasoning ability of LLMs by incorporating logical prompts into the model.
- Evaluation on theoretical problems shows that L3M outperforms traditional LLMs in solving logical equations.
- We also show the effectiveness of L3M on practical recommendation tasks, demonstrating its potential for a wider scope of applications.

In the following parts of the article, we will discuss the related work in Section 2, introduce the proposed model in Section 3, analyze the experimental results in Section 4, and provide the conclusions as well as future work in Section 5.

2 Related Work

LLMs. LLMs have gained significant attention in recent years due to their remarkable performance on various tasks, such as NLP [4, 6, 66], computer vision [13, 30, 42], and recommender systems [17]. The Transformer architecture [51] has become widely adopted for language modeling due to its ability to handle long-range dependencies in sequential data. For instance, the Transformer model [12, 51] was trained with billions of parameters and achieved state-of-the-art performance on multiple benchmarks. GPT-2 [39] demonstrated a major leap forward by efficiently handling various

language tasks without task-specific fine-tuning. This approach was further advanced by GPT-3 [4] and ChatGPT-4 [1], which improved on the scale and performance, setting new benchmarks in language model capabilities.

Parallel to these developments, models like T-5 [41] and LLaMA [50] provide flexibility for fine-tuning on diverse tasks. These models showcase the varied approaches within the field, each with unique strengths tailored to different challenges in NLP. This diversity not only enriches the toolkit available to researchers but also underscores the continuous innovation driving the technology forward.

Recent research has emphasized the importance of logical reasoning in LLMs [3, 56, 62]. LLMs that incorporate reasoning have shown improvements in solving mathematical problems [8, 16, 20, 54, 55, 69] and problem decomposition [14, 15, 28]. In addition, LLMs have been integrated into recommendation systems [11, 17, 48, 59–61, 70] to capture complex user behavior patterns and model the interactions between users and items, leading to improved performance on recommendation tasks. However, despite the advancements in LLMs, logical reasoning still remains a major weakness and requires further research efforts [9, 40].

Logical Reasoning in AI. Logical reasoning in AI has long been an important topic that can be traced back to 1943, according to McCulloch and Pitts [36]. In recent years, Markov logic networks have emerged as a promising approach to bridge logical reasoning and machine learning [38, 43, 68]. These models incorporate probabilistic graphical models and first-order logic and have achieved impressive results. However, not all practical problems are well-suited for hard-rule logical reasoning. For example, researchers have shown that equation solvers like PicoSAT [2] are effective on clean, non-contradictory logical equation problems. However, when it comes to reasoning with visual images, the model struggles to accurately identify objects, as demonstrated by Yi et al. [64]. This highlights the limitations of traditional logical reasoning in more complex, real-world situations.

To tackle the challenge of reasoning tasks that contain contradictions in the data, researchers have combined the power of neural networks with logical reasoning. NLR [46] trains basic logic operators as neural modules and achieves state-of-the-art accuracy on solving logical equation problems. Meanwhile, NCR [5] improves the encoding method and shows improved performance on recommendation tasks. Recent advancements in AI have highlighted the capabilities of LLMs in tackling complex logical reasoning problems [9, 10, 19, 23–25]. Studies have demonstrated that LLMs can effectively perform tasks that require deductive reasoning [44], problem-solving [63], and decision-making [31]. The ability to process and generate coherent, contextually appropriate responses makes them invaluable tools in fields ranging from automated customer service [29] to complex decision support systems [52]. Furthermore, ongoing research continues to refine these models, enhancing their efficiency and expanding their applicability in more rigorous logical frameworks.

Prompt-Based Learning. The advent of pre-trained LLMs has spurred the increased utilization of prompts in NLP tasks. In traditional NLP systems that employ supervised learning, a model $P(\mathbf{y}|\mathbf{x}; \theta)$ is utilized to predict the output $\mathbf{y}$ based on the input $\mathbf{x}$. However, the reliance on ample labeled training data can pose a significant challenge for certain NLP tasks [32]. To overcome this challenge, prompt-based learning methods, such as GPT-3, provide a set of input–output examples as prompts. Prompt-based learning has emerged as a pivotal approach in training LLMs [26, 58], enhancing their adaptability and efficiency in handling a diverse array of tasks. This methodology leverages the generalization capabilities of LLMs by utilizing prompts to guide the model’s response generation, effectively applying its pre-trained knowledge without necessitating extensive task-specific datasets. The inherent flexibility of prompt-based learning allows for a significant reduction in the need for additional training data and computational resources [35], which are typically required for traditional fine-tuning methods. In this regard, we present a novel

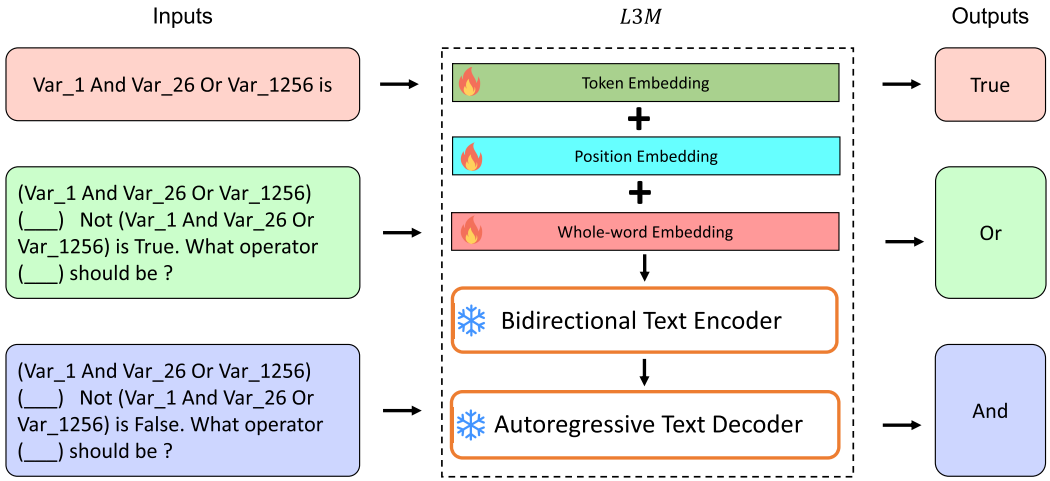


Fig. 2. The L3M utilizes an encoder–decoder structure. It begins by encoding the input using a bidirectional text encoder and then generates the output via an autoregressive text decoder. L3M takes into account both the original sequence and the sequence generated by logical prompts during the training process. This enables the model to learn the rules of each logic operator.

approach where we create a collection of logical prompts and train a sequence-to-sequence model. This approach demonstrates remarkable performance in solving both theoretical problems, such as logical expressions, and practical problems, such as recommendations.

3 L3M

In this section, we present our proposed L3M. Our model represents logical expressions as natural language sequences, enabling the language model to learn and infer the relationships between these expressions. Unlike current LLMs, which face challenges in solving complex logical equations due to limited training data, our L3M leverages the use of logical prompts to enhance its inferential capabilities. This innovative approach overcomes the limitations of conventional LLMs and demonstrates the potential for improved performance on logical reasoning tasks.

3.1 L3M Architecture

In this section, we present our L3M, which is built upon an encoder–decoder architecture. The encoder and decoder components of L3M are constructed using Transformer blocks [51]. The Transformer architecture has gained popularity for NLP tasks due to its ability to handle long-range dependencies effectively. The encoder in L3M processes the input sequence and transforms it into a fixed-length vector representation, while the decoder generates the output sequence based on the encoded input.

As depicted in Figure 2, L3M leverages the architecture of the T5 backbone and retains the bidirectional text encoder and autoregressive decoder. To improve its performance, L3M incorporates three types of embeddings: token embedding, position embedding, and whole-word embedding. These embeddings provide additional information to the model, with token embedding representing individual words, position embedding indicating the position of each word in the input sequence, and whole-word embedding capturing the contextual information of a word. The input sequence undergoes these three embedding processes, and the results are then combined and fed into the bidirectional text encoder, which encodes the input sequence. This enables the model to have a more comprehensive understanding of the input, resulting in improved prediction accuracy.

Suppose the embeddings of an input token sequence is denoted as $\mathbf{e} = [e_1, e_2, \dots, e_n]$. As illustrated in Figure 2, we enhance the raw embeddings with positional encoding to incorporate information about the position of the tokens in the sequence before feeding them into the bidirectional text encoder [12]. Furthermore, we leverage whole-word embeddings to facilitate L3M's ability to recognize consecutive sub-word tokens that belong to the same original word [17]. For instance, a "long ID" such as "Var_1256" would be segmented into four tokens ("Var," "_," "12," and "56") by the SentencePiece tokenizer [45]. As a result, the four tokens will be added by the same whole-word embedding to indicate that they come from the same word in the original sequence. The use of whole-word embeddings allows L3M to more accurately identify fields containing personalized information.

The bidirectional text encoder combines the three aforementioned embeddings, resulting in the contextual representations $\mathbf{x} = [x_1, x_2, \dots, x_n]$. The decoder, in turn, utilizes both the previously generated tokens $\mathbf{y}_{i < j}$ and the encoder's output $\mathbf{x}$ to predict the probability distribution of future tokens, as expressed by the equation $P_\theta(\mathbf{y}_j | \mathbf{y}_{i < j}, \mathbf{x})$. During the pre-training phase, the model learns the parameters θ by minimizing the negative log-likelihood of the label tokens $\mathbf{y}$ given the input text $\mathbf{x}$:

$$\mathcal{L} = - \sum_{j=1}^{|\mathbf{y}|} \log P_\theta(\mathbf{y}_j | \mathbf{y}_{i < j}, \mathbf{x}). \quad (2)$$

This negative log-likelihood loss function is applied to both theoretical and practical tasks in the pre-training phase of L3M.

3.2 Logical Prompts

So far, we have equipped L3M with the ability to represent logical expressions as natural language sequences. However, simply formatting the expressions does not guarantee that the model has a proper understanding of the logic operators, such as And, Or, and Not. For instance, it is known that a double negation of any logical expression should result in itself ($\neg\neg x = x$). While it is possible for the model to implicitly learn these operations through training data, a more effective approach is to provide additional guidance through the use of constraints. To this end, we introduce the use of logic prompts to regulate the learning process of the model. These prompts serve to ensure that the model understands and adheres to the rules of the logic operators. A comprehensive list of these prompts can be found in Table 1.

The logic prompts are grouped into three categories, corresponding to the logic operations. To formulate these prompts, the logical expressions are transformed into natural language sequences. For instance, applying the "And Annihilator" prompt to the statement "Var_1 And Var_26 Or Var_1256 is True" results in "(Var_1 And Var_26 Or Var_1256) And False is False," where "Var_1 And Var_26 Or Var_1256" is the input applied to the prompt.

It is important to note that, in addition to the listed logic prompts, a propositional logical system must also adhere to other logical rules, such as associativity, commutativity, and distributivity of And, Or, and Not operations. For example, commutativity of the Or operation requires $a \vee b = b \vee a$. To account for commutativity, if a logical rule has two distinct variables on the left side of the equality sign, both scenarios are considered to ensure thorough training of the model. For instance, in the "Or Annihilator" prompt, both $x \vee T = T$ and $T \vee x = T$ are considered, encouraging the model to produce the same output for different forms of the same expression due to the commutativity.

While enforcing commutativity can be easily implemented by simple prompts that consider both directions as shown above, it is challenging to explicitly regularize for associativity and distributivity since it will result in over-complicated prompts. For simplicity, this study only

Table 1. Logical Rules and the Corresponding Logical Prompts

	Logical Rule	Equation	Logical Prompts
Not	Negation	$\neg T = F$	Not True is False
		$\neg F = T$	Not False is True
	Double Negation	$\neg\neg x = x$	Not Not {input} is {input}
And	Identity	$x \wedge T = x$	{input} And True is {input}
		$T \wedge x = x$	True And {input} is {input}
	Annihilator	$x \wedge F = F$	{input} And False is False
		$F \wedge x = F$	False And {input} is False
	Idempotence	$x \wedge x = x$	{input} And {input} is {input}
	Complementation	$x \wedge \neg x = F$ $\neg x \wedge x = F$	{input} And Not {input} is False Not {input} And {input} is False
Or	Identity	$x \vee F = x$	{input} Or False is {input}
		$F \vee x = x$	False Or {input} is {input}
	Annihilator	$x \vee T = T$	{input} Or True is True
		$T \vee x = T$	True Or {input} is True
	Idempotence	$x \vee x = x$	{input} Or {input} is {input}
	Complementation	$x \vee \neg x = T$ $\neg x \vee x = T$	{input} Or Not {input} is True Not {input} Or {input} is True

enforces the commutativity property by prompts. To overcome the challenge of associativity and distributivity, following Shi et al. [46], we guarantee that all inputs are transformed into **Disjunctive Normal Form (DNF)** to simplify the model's learning process, since any logical expression can be transformed into DNFs. We will show that this approach already suffices for our model to outperform many state-of-the-art models.

In the learning process, the model not only predicts the True or False values of logical prompts but also the relationships between inputs. As demonstrated in Figure 2, the L3M is trained to predict the missing operator in logical prompts such as “And Complementation.”

3.3 The Logical Equation Solving Task

In the previous section, we structured the logical equations and devised appropriate prompts for the three logic operations. Before we proceed to practical tasks, we will first evaluate the logical reasoning ability of our model using a simulated dataset on the theoretical task of solving logical equations.

Previous efforts to use LLMs for this task, as reported in [47], have not resulted in satisfactory performance due to the lack of reasoning capability. However, with the use of L3M, we aim to overcome this limitation and achieve improved performance on the task of solving logical equations. The simulated dataset will consist of only DNF logical equations, as any propositional logical statement can be transformed into DNF.

During the training phase, our approach with the model involves a significant deviation from conventional methods. Typically, models are trained with explicit indicators of “true” or “false” for each variable in logical equations. However, in our methodology, we deliberately withhold these

User_id: 125

Interaction history: Camera, Camera Lens, Memory Card

Recommendation Prompt Example:

Input: For use_125: Camera And Camera Lens And (___) is True. What is (___)?

Output: Memory Card.

Logical Prompt Example:

Input: For use_125: (Camera And Camera Lens And Memory Card) (___) False is False. What operator (___) should be?

Output: And.

Input: For use_125: (Camera And Camera Lens And Memory Card) (___) True is True. What operator (___) should be?

Output: Or.

Fig. 3. L3M on recommendation task. Based on the user ID and corresponding purchase history, L3M can convert them to logical equations.

explicit truth values from the model. This means that the L3M will not receive direct information about the correctness of each variable within the logical equations.

Instead, L3M is expected to utilize its intrinsic reasoning abilities to infer the values of the variables. It will have to analyze the logical operations present in each equation—be it AND, OR, or NOT—and use this understanding to deduce whether a variable stands True or False in the given context. This approach fundamentally shifts the model’s reliance from pre-fed information to its own logical deduction capabilities.

In summary, our training process for L3M is designed to challenge the model beyond conventional parameters. By compelling it to rely on its own logical reasoning abilities rather than on provided answers, we aim to not only test its current capabilities but also push its boundaries in logical deduction. This approach is expected to provide a deeper, more nuanced understanding of the model’s performance in logical reasoning tasks.

3.4 The Recommendation Task

In this study, we aim to evaluate the performance of L3M in solving practical problems by conducting experiments on sequential recommendation tasks. The task of recommendation systems is to understand user preferences based on their past interactions, which can be modeled effectively using logical expressions as demonstrated in previous works by Shi et al. [46] and Chen et al. [5].

As depicted in Figure 3, we present an example of the recommendation and logical prompts for a user. To convert the recommendation task into a problem that can be solved by L3M, we translate it into a logical expression. In the example, given a user’s purchase history of purchasing a camera, a camera lens, and a memory card, we represent the relationship between the items as a logical equation and leave the latest purchase history as a blank for the model to predict.

The logical prompts can also be adapted for the sequential recommendation task by modifying the inputs to represent a user’s purchase history. As an example, consider the scenario where a user has purchased a camera, followed by a camera lens and memory card. The logical prompt “And Annihilator” can be applied to this sequence to yield the expression “For user_125: camera And camera lens And memory card (___) False is False.” L3M is then tasked with predicting the appropriate logical operator. Due to the limited input token length during the training phase, the item names are transformed into item IDs, such as “item_1234” for camera and “item_360” for camera lens.

4 Experiment

In this section, we present the empirical evaluation of the L3M approach. Our experiments are divided into two parts: the first part focuses on the evaluation of L3M on a simulated dataset for solving logical equation tasks, while the second part evaluates the model's performance on real-world datasets for recommendation tasks. To provide a comprehensive assessment of L3M, we compare its performance to various representative methods in both parts of the experiments. The results show that our proposed approach outperforms the compared methods significantly on both simulated and real-world datasets.

4.1 Dataset

4.1.1 Logical Equation Solving. The initial phase of our experiments focused on assessing the proposed L3M framework's capacity for logical inference within a simulated task. This task was specifically designed to challenge traditional LLMs and to showcase L3M's enhanced capabilities in logical reasoning. For training purposes, we utilized a dataset composed of logical expressions paired with their respective True/False values. The evaluation of the model was conducted by testing its ability to deduce the True/False values of the logic variables and to predict the outcomes of unseen expressions that were constructed using the observed logic variables.

The dataset was generated by randomly selecting n variables, $\mathcal{V} = \{v_i\}_{i=1}^n$, and assigning each variable a random value of True or False. These variables were then used to randomly generate m Boolean expressions $\mathcal{E} = \{E_j\}_{j=1}^m$ in DNF as the dataset. The True/False values of the expressions, $\mathcal{Y} = \{y_j\}_{j=1}^m$, can be determined based on the variables. However, it is important to note that the True/False values of the variables were not made visible to the model during the training process. The following examples illustrate a few of the generated expressions when $n = 100$:

$$\begin{aligned} &(\neg v_{32} \wedge v_{44}) \vee (\neg v_{12} \wedge \neg v_{78} \wedge v_5 \wedge \neg v_{11}) \\ &\quad \vee v_{12} \vee (v_2 \wedge v_{33} \wedge v_{67} \wedge v_{55} \wedge v_{15}) = T \\ &\neg v_{24} \vee (v_{25} \wedge \neg v_{73} \wedge v_{64} \wedge \neg v_{52} \wedge v_{57}) \\ &\quad \vee (v_{17} \wedge v_{62}) \vee (\neg v_8 \wedge \neg v_{39} \wedge \neg v_{41} \wedge v_{63}) = F \\ &v_{43} \vee (\neg v_{25} \wedge v_{17} \wedge v_{54}) \vee (v_{32} \wedge v_{41} \wedge v_{37} \wedge v_{29}) = T. \end{aligned}$$

In this experiment, the L3M is trained using a dataset of known logical equations, where the goal is for the model to deduce the True/False values of unseen equations. In this experiment, we employed two simulated datasets. The first dataset consists of $n = 1,000$ variables and $m = 3,000$. The second dataset contained $n = 3,000$ variables and $m = 9,000$. Both of them are formatted in DNF. To ensure an unbiased evaluation, the dataset is split into three parts: 80% for training, 10% for validation, and 10% for testing. The T/F values of the variables are not made visible to the model during training.

4.1.2 Recommendation Dataset. We conduct extensive experiments on three real-world Amazon¹ datasets to evaluate the performance of our proposed L3M approach on recommendation tasks, namely, Sports and Outdoors, Beauty, and Toys and Games. The datasets were collected from the Amazon.com platform and include user purchase history across various products. The statistics of these datasets are presented in Table 2. For each user interaction sequence, the most recent item is used as the test data, the second-most recent is used as validation data, and the remaining is used for training.

¹<https://nijianmo.github.io/amazon/>.

Table 2. Basic Statistics of the Recommendation Datasets

Dataset	Sports	Beauty	Toys
#Users	35,598	22,363	19,412
#Items	18,357	12,101	11,924
#Sparsity (%)	0.0453	0.0734	0.0724

4.2 Implement Details

The proposed L3M model utilizes the T5 and LLaMA checkpoint [41, 50] as its backbone. The L3M-T5 consists of six layers in both the encoder and decoder, which makes a balance between model complexity and computational efficiency. Each layer is designed to capture rich semantic representations through its 8-headed attention mechanism. And the model dimensionality is set to 512, ensuring sufficient capacity for learning contextualized embeddings while maintaining scalability for training and inference. For L3M-LLaMA, we used LoRA [22] and set the rank as 32 to optimize efficiency while maintaining comparable performance. The input is tokenized using the SentencePiece [45] tokenizer, which segments the input into sub-word units with a vocabulary size of 32,128.

The L3M model was pretrained for 20 epochs using the AdamW optimization [33] on four NVIDIA RTX A5000 GPUs with a batch size of 16. The peak learning rate was set to 1×10^{-3} and the maximum input length was set to 512 tokens. A warm-up strategy was employed during training, where the learning rate was gradually increased over the first 5% of all iterations. For the recommendation task, to prevent the model from over-relying on logical prompts, the ratio of logical prompts used in training was balanced. For each user, a random number between [0,1] was generated, and if it is greater than the set threshold, a recommendation prompt is used to process the user history into a training instance, otherwise, a logical prompt is used. We use grid search in [0.001,0.01,0.1,0.2,0.5] to find the best threshold for each dataset, and we will show the model performance under different ratios in the experiments. The baselines were run using the parameter configurations specified in their original papers.

4.3 Evaluation Metrics

4.3.1 Logical Equation Solving. For the Logical Equation Solving task, we use three standard evaluation metrics: accuracy, precision, and recall, to evaluate the performance of solving the True/False results for the testing logical expressions. The accuracy metric assesses the proportion of correct predictions made by the model among all predictions. Precision measures the ratio of true positive predictions among all positive predictions made by the model, while recall evaluates the proportion of true positive predictions among all actual positive cases. These three metrics provide a comprehensive evaluation of the model's performance.

4.3.2 Sequential Recommendation Task. For the Sequential Recommendation task, we evaluate the performance of the model using two commonly used metrics in the literature: **Hit Ratio (HR)** and **Normalized Discounted Cumulative Gain (NDCG)**. The HR metric determines the proportion of recommended items that are present in the ground truth, while NDCG measures the effectiveness of the ranked recommendations taking into account the relevance of the recommended items. Both metrics are widely used to evaluate recommendation systems.

4.4 Baseline Methods

We introduce the compared baseline methods for the logical equation solving task and the sequential recommendation task, respectively.

4.4.1 Logical Equation Solving. In the logical equation solving experiments, we compare the performance of our L3M model with three state-of-the-art baselines: two NLR models and an LLM.

- *NLR* [46]: The NLR model presents a Logic Integrated Neural Network, which combines the benefits of both logical reasoning and neural networks to enhance its performance.
- *NCR* [5]: The NCR model introduces the concept of logical regularizers, which can enhance the model’s ability to learn the logical relationship among various variables.
- *T5* [41]: T5 is a LLM developed by Google based on the transformer architecture. The model has been pre-trained on a diverse range of texts, which enables it to perform a variety of text generation tasks.
- *ChatGPT-4*: ChatGPT-4 is a state-of-the-art language model developed by OpenAI. This model is trained on a vast and varied dataset, equipping it to handle a wide array of language-based tasks, from complex text generation to nuanced dialogue interactions.

4.4.2 Sequential Recommendation. In the recommendation tasks, several widely used sequential recommendation models were utilized as baselines for comparison with our proposed approach.

- *Convolutional Sequence Embedding Recommendation (Caser)* [49]: The Caser model employs a Convolutional Neural Network architecture to capture the intricate sequential patterns inherent in high-order Markov chains.
- *Hierarchical Gating Network (HGN)* [34]: The HGN approach leverages hierarchical gating mechanisms to effectively differentiate between long-term and short-term user preferences, resulting in more accurate recommendations.
- *GRU4Rec* [21]: The GRU4Rec model is a session-based recommendation approach that employs a Gated Recurrent Unit to capture user preferences. The model leverages previous preferences within a session as context to make predictions regarding the next item that a user is likely to engage with.
- *BERT4Rec* [48]: The BERT4Rec approach to sequential recommendation extends traditional **Recurrent Neural Network (RNN)** methods by incorporating a Cloze objective and a bidirectional self-attention mechanism. The use of bidirectional self-attention further enhances the model’s ability to capture the inter-dependencies between items in the sequence.
- *Feature-Level Deeper Self Attention (FDSEA)* [67]: The FDSEA model employs a multi-feature extraction technique to construct a feature sequence from the items in a given sequence. This enables the model to capture the dynamic interplay of various features and to weigh their relative importance in the prediction process.
- *Self-Attentive Sequential Recommendation (SASRec)* [27]: The SASRec approach integrates the self-attention mechanism into a sequential recommendation model, effectively balancing the properties of Markov Chains and RNN-based approaches.
- *S³-Rec* [70]: The self-supervised learning for sequential recommendation (S³-Rec) approach leverages self-supervised objectives to help sequential recommendation model better discover the correlations among different items and their attributes, resulting in improved performance.
- *P5* [17]: P5 employs a variety of templates for the input and target sequences during the training process, effectively breaking down the distinction between various tasks.
- *LLaMA* [50]: LLaMA is designed to provide exceptional flexibility for fine-tuning on a wide variety of tasks, making them adaptable to different domains and applications. Their

Table 3. Performance Comparison on Logical Equation Solving

	logic 1K			logic 3K		
	Accuracy	Precision	Recall	Accuracy	Precision	Recall
NLR	0.903	0.907	0.921	0.897	0.901	0.913
NCR	0.906	0.912	0.925	0.901	0.906	0.917
T5	0.873	0.879	0.929	0.869	0.873	0.922
ChatGPT-4	0.657	0.672	0.685	0.632	0.657	0.664
L3M	0.913*	0.919*	0.934*	0.910*	0.913*	0.928*

Bold numbers represent best performance. We use * to indicate that the performance is significantly better than baselines. The significance is at 0.05 level on paired *t*-test.

architecture allows users to modify and optimize the models for specific use cases, from natural language understanding to more specialized tasks such as recommendation.

4.5 Performance Comparison on Logical Equation Solving

The performance of solving logical equations is shown in Table 3, where accuracy is calculated as the percentage of correctly classified equations as either True or False. Precision and recall are metrics that measure the proportion of true positive predictions to all positive predictions and actual positive cases, respectively. The higher the values of these three metrics, the better the performance of the model. In summary, the evaluation results in Table 3 show that our proposed L3M outperforms the baselines in terms of accuracy, precision, and recall. These results highlight the efficacy of our approach and suggest its potential for solving complex logical reasoning problems.

It is noteworthy that when evaluating T5's performance on logical equation solving tasks, the model demonstrated inferior results in comparison to the much smaller NLR model. This can be attributed to the absence of logical prompts in T5, as logical reasoning is a critical aspect in numerous theoretical and practical problems. Despite having a much larger number of parameters than NLR and NCR, the model's lack of logical reasoning abilities limits its performance. This emphasizes the significance of incorporating logical prompts or alternative methods to facilitate the development of logical reasoning in these models, in order to enhance their performance on related tasks.

To delve deeper into the capabilities of different models, we also conducted extensive tests on ChatGPT-4. Our findings reveal that ChatGPT-4 achieves an accuracy rate of approximately 65% in this task. This performance metric is notably inferior when compared to other baseline models used in our study. The reason for this disparity in performance could be attributed to the inherent limitations associated with the training of ChatGPT-4. Unlike other models that allow for customization through pre-training or fine-tuning on specific tasks, ChatGPT-4 does not support such modifications post its initial training phase. This restriction potentially hinders its ability to adapt and optimize its responses for tasks that were not sufficiently covered during its training. Consequently, this could lead to a decreased proficiency in handling tasks such as logical equation solving, which demands a high degree of specialized knowledge and adaptability.

The performance of the L3M model on the logical equation solving task is significantly improved by incorporating logical prompts. As evidenced by the results shown in the evaluation, the L3M model outperforms T5, NLR, and ChatGPT-4 in terms of accuracy, precision, and recall. This outcome indicates that the utilization of logical prompts effectively enhances the logical reasoning capabilities of L3M. These findings suggest that the incorporation of logical prompts may be a

promising approach for enhancing the logical reasoning ability of LLMs in various applications.

Correct prediction case of L3M

$$(v_{58} \vee v_7 \vee \neg v_{16}) \wedge (\neg v_2 \wedge \neg v_{74}) \wedge v_{28} \wedge v_{13} \vee v_{69} = T$$

$$v_{34} \vee v_{42} \wedge \neg v_{91} \wedge v_{63} = T$$

Wrong prediction case of L3M

$$(v_3 \vee v_{11}) \vee v_{15} \wedge (\neg v_{78} \wedge v_5 \vee v_{11}) \wedge v_{18} \vee (v_{32} \wedge v_{56} \vee \neg v_{86} \vee v_{75} \vee v_{26}) = T$$

$$(\neg v_{16} \wedge v_2 \wedge v_8 \vee \neg v_{52}) \wedge v_{37} \vee v_{82} \vee v_{69} \wedge v_{23} \vee (v_{19} \wedge \neg v_{21} \wedge \neg v_{29}) = F$$

Correct prediction case of GPT-4

$$v_{26} \wedge v_{31} \wedge \neg v_{93} \wedge v_{49} = T$$

$$v_{37} \vee v_{26} \wedge v_{71} = T$$

Wrong prediction case of GPT-4

$$\neg v_{59} \wedge \neg v_{71} \wedge v_9 \vee \neg v_{63} \wedge v_{84} = T$$

$$v_{21} \vee \neg v_{45} \wedge (v_{16} \vee \neg v_{96} \wedge \neg v_{49}) \vee (v_{19} \wedge v_{28} \wedge \neg v_{64}) = F$$

We compared the incorrect predictions made by our L3M model with those made by ChatGPT-4. Our findings reveal distinct patterns of failure for the two models. In most cases, L3M produces incorrect results when the logical equation is lengthy, whereas ChatGPT-4 performs poorly on both short and long logical equations.

The superior performance of L3M on shorter equations can be attributed to the use of logical prompts, which help the model better understand the variables and logical operators within the equation. However, when the equation becomes too long, the corresponding logical prompts also grow in complexity, making it harder for L3M to interpret and process the equation accurately. In contrast, ChatGPT-4 does not have logical prompts, which results in consistently poor performance regardless of the equation's length. This highlights the importance of incorporating structured prompts to guide the model's reasoning process, particularly for tasks involving complex logical structures. When experimenting with more general datasets [18], where it is difficult to apply logical prompts, the performance of ChatGPT (ROUGE-L: 0.343) and L3M (ROUGE-L: 0.416) is more comparable. This also emphasizes the importance of logical prompts.

4.6 Performance Comparison on Recommendation

The results of the experiment on the NDCG and HR metrics are presented in Table 4. Our observations are as follows.

Upon comparing our L3M model to the baselines, it was observed that the former outperformed the latter on all three datasets. These experimental results showcase the potential of the L3M model in handling practical tasks effectively.

It is worth mentioning that the results of both our L3M models on NDCG and HR metrics show significant improvement compared to traditional sequential recommendation models such as S^3 -Rec and SASRec, as depicted in Table 4. The utilization of a transformer architecture in our model enables it to effectively capture long-term dependencies and relationships between items, which is vital in sequential recommendation tasks. Additionally, the capacity of LLM to process larger amounts of data allows for better learning and generalization of related items for a particular user.

In comparison to the LLM model P5 and LLaMA, our L3M shows superior performance on recommendation tasks. This improvement is attributed to the integration of logical prompts in L3M, which enables the model to perform logical reasoning and identify concrete connections between users and items. This results in a more informed decision-making process and a better

Table 4. Experimental Results on NDCG (NDCG@k) and HR (HR@k)

Methods	Sports				Beauty				Toys			
	HR@5	NDCG@5	HR@10	NDCG@10	HR@5	NDCG@5	HR@10	NDCG@10	HR@5	NDCG@5	HR@10	NDCG@10
Caser	0.0116	0.0072	0.0194	0.0097	0.0205	0.0131	0.0347	0.0176	0.0166	0.0107	0.0270	0.0141
HGN	0.0189	0.0120	0.0313	0.0159	0.0325	0.0206	0.0512	0.0266	0.0321	0.0221	0.0497	0.0277
GRU4Rec	0.0129	0.0086	0.0204	0.0110	0.0164	0.0099	0.0283	0.0137	0.0097	0.0059	0.0176	0.0084
BERT4Rec	0.0115	0.0075	0.0191	0.0099	0.0203	0.0124	0.0347	0.0170	0.0116	0.0071	0.0203	0.0099
FDSA	0.0182	0.0122	0.0288	0.0156	0.0267	0.0163	0.0407	0.0208	0.0228	0.0140	0.0381	0.0189
SASRec	0.0233	0.0154	0.0350	0.0192	0.0387	0.0249	0.0605	0.0318	0.0463	0.0306	0.0675	0.0374
S ³ -Rec	0.0251	0.0161	0.0385	0.0204	0.0387	0.0244	0.0647	0.0327	0.0443	0.0294	0.0700	0.0376
P5	0.0387	0.0312	0.0460	0.0336	0.0493	0.0367	0.0645	0.0416	0.0587	0.0486	0.0675	0.0536
llama	0.0452	0.0371	0.0479	0.0332	0.0551	0.0397	0.0596	0.0421	0.644	0.563	0.658	0.519
L3M-T5	0.0442	0.036	0.0514*	0.0393*	0.0545	0.0415	0.0697*	0.0465*	0.0631	0.0557	0.0708*	0.0582*
L3M-LLaMA	0.04769*	0.03849*	0.0503	0.0363	0.05789*	0.04219*	0.0615	0.0433	0.6539*	0.5839*	0.692	0.553

Bold numbers represent best performance.

*Indicates that the performance is significantly better than baselines. The significance is at 0.05 level on paired *t*-test.

understanding of the user's needs and future preferences, thereby improving the performance of the model. Since we cannot pre-train or fine-tune ChatGPT, the recommendation results of ChatGPT are not included in the present study.

Our framework is adaptable and can be applied not only to the encoder-decoder model T5 but also to the decoder-only model LLaMA. We developed two versions, L3M-T5 and L3M-LLaMA, both of which outperform their respective base models in various performance metrics. When comparing the two, L3M-LLaMA demonstrates superior performance in HR@5 and NDCG@5, showing its strength in top-5 rankings. On the other hand, L3M-T5 excels in HR@10 and NDCG@10, indicating its effectiveness in broader top-10 recommendations.

4.7 Impact of Different Backbones

In this detailed section, we evaluate an important aspect of our research: analyzing how the size of the model influences the effectiveness of the L3M in various recommendation tasks. Our investigation centers on two distinct variants of the T5 model, namely T5-small and T5-base. These models serve as the backbone for our L3M variations, specifically designed for this study: L3M-small and L3M-base. The T5-small model, with its relatively modest 60.75 million parameters, offers a more compact framework. In contrast, T5-base is significantly larger, encompassing a substantial 223.28 million parameters.

A critical part of our analysis involves comparing the performance of these two L3M versions across a range of recommendation tasks and datasets. Remarkably, as illustrated in Figure 4, the results reveal a somewhat counterintuitive trend. Despite its smaller size, which represents approximately one-fourth of the parameter count of L3M-base, L3M-small consistently outperforms L3M-base across all evaluated tasks and datasets. This finding is particularly intriguing as it challenges the commonly held belief that larger models, with their extensive parameter sets, inherently lead to better performance.

One possible reason for this unexpected outcome lies in the concept of parameter efficiency. The T5-small model, which forms the foundation of L3M-small, may benefit from a kind of pruning effect. This process effectively eliminates redundant or less significant parameters within the model. Such a refinement not only streamlines the model but also potentially enhances its focus and efficiency in learning and applying relevant patterns and rules specific to our recommendation tasks.

It's important to consider the broader context of our work in light of recent trends in LLM development, which have largely favored decoder-only models such as LLaMA [50]. Although our

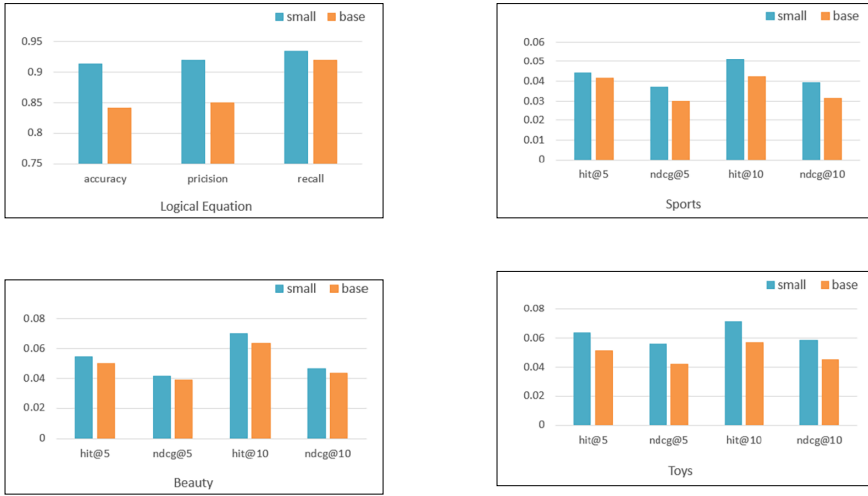


Fig. 4. Performance of L3M-small and L3M-base on solving logical equations and sequential recommendation.

logical prompts were initially designed for an encoder–decoder setup, they can indeed be applied to other current methods; however, their effectiveness might be affected. This adaptability underscores the potential of our techniques to contribute across different model architectures, further expanding the practical utility and applicability of our research in the evolving landscape of LLMs.

The implications of these findings are significant. They point towards a potential way on how we approach the design and scaling of AI models, particularly in the context of recommendation systems and similar applications. Our research here lays the groundwork for further exploration into this domain, encouraging a more nuanced understanding of how model size and architecture can be optimized to achieve superior performance.

4.8 Impact of the Ratio of Logical Prompts

In this section, we evaluate the effect of the threshold that adjusts the ratio of logical prompts on the performance of L3M. In the implement details we mentioned that we will generate a random number between 0 and 1 to determine whether to choose recommendation prompt or a logical prompt to convert a user’s interaction sequence into a training sentence (for logical equation solving, recommendation prompt is the logical expression itself). Specifically, we will set a ratio threshold and if the random number is bigger than the ratio, a recommendation prompt will be selected, otherwise a logical prompt will be selected. We present the experimental results in Figure 5 for both simulated and real-world data. Our results show that L3M improves its performance with increasing threshold initially, reaching its peak at a threshold of approximately 0.1 to 0.2. However, as the threshold continues to rise, the performance begins to decline.

It is important to note that a very low threshold leads to a slightly decreased performance since the small number of logical prompts have limited impact on the model’s parameters. On the other hand, a very high threshold also results in decreased performance since the dominance of logical prompts in the training process hampers the model’s ability to learn from the recommendation prompts.

4.9 Impact of the Scope of Logical Prompts

In this study, we aim to evaluate the impact of the scope of logical prompts on the performance of our L3M model. According to Table 1, we have 10 logical prompts in total. We explore what

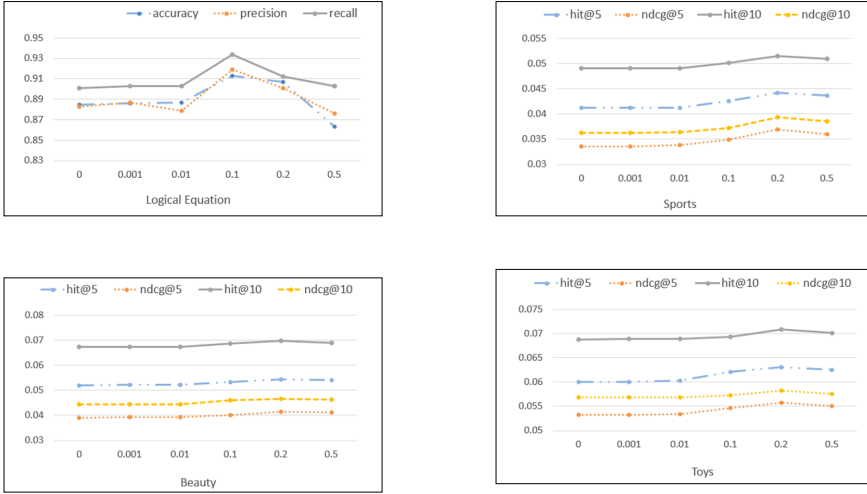


Fig. 5. L3M performance on different datasets under different ratios of logical prompts.

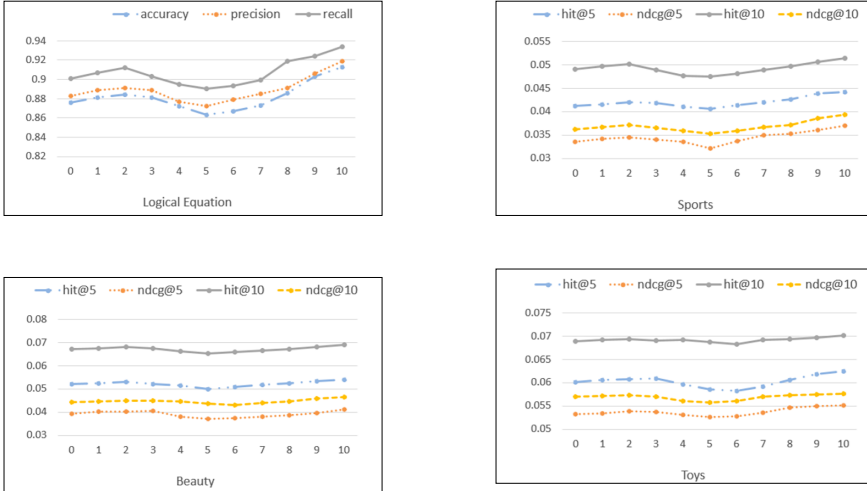


Fig. 6. L3M performance on different datasets with different scopes of logical prompts.

happens if we only consider the first k logical prompts. For example, when $k = 3$, we only consider the negation, double negation, and And-Identity logical prompts. We conduct experiments by varying the scope of logical prompts k from 0 to 10 and present the experimental results in Figure 6. Our findings, based on both simulated and real-world data, indicate that a small scope of logical prompts can improve the performance of the L3M model. However, as the scope of logical prompts increases, the performance begins to decline and reaches its lowest point when around five logical prompts are used. Subsequently, as the scope of logical prompts increases further, the performance improves again until reaching its peak when all the logical prompts are utilized.

The observed decline in performance during the middle stage of the process can be attributed to the introduction of some but not all logical prompts, which causes more confusions than benefits to the model, leading to suboptimal results. However, when the model receives all of the logical prompts, it can better comprehend the underlying rules, leading to improved performance.

5 Conclusions

In this study, we introduced L3M, a novel approach that enhances the capabilities of LLMs with the precision of logical reasoning. Our comprehensive experimentation on simulated data underscores the remarkable efficiency of L3M in tackling theoretical challenges, notably in solving complex logical equations. This approach is not limited to theoretical realms; our application of L3M on real-world datasets in recommender systems demonstrates its practical utility, showcasing its adaptability and effectiveness in addressing real-life problems.

Moreover, the integration of logical reasoning in L3M not only enhances its problem-solving capabilities but also contributes to a more nuanced understanding of decision-making processes. The use of logical prompts in the model architecture enables a structured and interpretable approach to problem-solving, distinguishing L3M from traditional LLMs in terms of clarity and reasoning proficiency.

Our future work aims to delve deeper into the realm of logical reasoning within LLMs. We plan to investigate alternative methods for embedding logical reasoning into these models, exploring diverse approaches that could further refine their reasoning capabilities.

In summary, L3M marks a big step forward in AI. It successfully combines the vast learning abilities of LLMs with the important skill of logical thinking. L3M has shown great results in both theoretical tasks, like solving logical equations, and real-world uses, such as recommending products or services. This achievement signals the beginning of a new phase in AI, where our models don't just rely on massive amounts of data but also use logical reasoning. This research sets the stage for future developments, opening doors to more advanced, trustworthy, and understandable AI systems.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. arXiv:2303.08774. Retrieved from <https://arxiv.org/abs/2303.08774>
- [2] Armin Biere. 2008. PicoSAT essentials. *Journal on Satisfiability, Boolean Modeling and Computation* 4, 2–4 (2008), 75–97.
- [3] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A large annotated corpus for learning natural language inference. In *2015 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 632–642. DOI : <https://doi.org/10.18653/v1/D15-1075>
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *34th International Conference on Neural Information Processing Systems*, 1877–1901.
- [5] Hanxiong Chen, Shaoyun Shi, Yunqi Li, and Yongfeng Zhang. 2021. Neural collaborative reasoning. In *Web Conference 2021*, 1516–1527.
- [6] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. Palm: Scaling language modeling with pathways. arXiv:2204.02311. Retrieved from <https://arxiv.org/abs/2204.02311>
- [7] Yu-An Chung and James Glass. 2020. Generative pre-training for speech with autoregressive predictive coding. In *2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP '20)*. IEEE, 3497–3501.
- [8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. arXiv:2110.14168. Retrieved from <https://arxiv.org/abs/2110.14168>
- [9] Antonia Creswell, Murray Shanahan, and Irina Higgins. 2022. Selection-inference: Exploiting large language models for interpretable logical reasoning. arXiv:2205.09712. Retrieved from <https://arxiv.org/abs/2205.09712>
- [10] Wang-Zhou Dai, Qiuling Xu, Yang Yu, and Zhi-Hua Zhou. 2019. Bridging machine learning and logical reasoning by abductive learning. In *33rd Conference on Neural Information Processing Systems*.
- [11] Gabriel de Souza Pereira Moreira, Sara Rabhi, Jeong Min Lee, Ronay Ak, and Even Oldridge. 2021. Transformers4Rec: Bridging the gap between NLP and sequential/session-based recommendation. In *15th ACM Conference on Recommender Systems*, 143–153.

- [12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT '19)*. Jill Burstein, Christy Doran, and Tamar Solorio (Eds.), Association for Computational Linguistics, 4171–4186. DOI : <https://doi.org/10.18653/v1/n19-1423>
- [13] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2021. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*. Retrieved from <https://openreview.net/forum?id=YicbFdNTTy>
- [14] Andrew Drozdov, Nathanael Schärli, Ekin Akyürek, Nathan Scales, Xinying Song, Xinyun Chen, Olivier Bousquet, and Denny Zhou. 2022. Compositional semantic parsing with large language models. arXiv:2209.15003. Retrieved from <https://arxiv.org/abs/2209.15003>
- [15] Dheeru Dua, Shivanshu Gupta, Sameer Singh, and Matt Gardner. 2022. Successive prompting for decomposing complex questions. arXiv:2212.04092. Retrieved from <https://arxiv.org/abs/2212.04092>
- [16] Yao Fu, Hao Peng, Ashish Sabharwal, Peter Clark, and Tushar Khot. 2022. Complexity-based prompting for multi-step reasoning. arXiv:2210.00720. Retrieved from <https://arxiv.org/abs/2210.00720>
- [17] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (RLP): A unified pretrain, personalized prompt & predict paradigm (P5). In *16th ACM Conference on Recommender Systems (RecSys '22)*. Jennifer Golbeck, F. Maxwell Harper, Vanessa Murdock, Michael D. Ekstrand, Bracha Shapira, Justin Basilico, Keld T. Lundgaard, and Even Oldridge (Eds.), ACM, 299–315. DOI : <https://doi.org/10.1145/3523227.3546767>
- [18] Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. 2019. SAMSum corpus: A human-annotated dialogue dataset for abstractive summarization. In *2nd Workshop on New Frontiers in Summarization*. Lu Wang, Jackie Chi Kit Cheung, Giuseppe Carenini, and Fei Liu (Eds.), Association for Computational Linguistics, 70–79. DOI : <https://doi.org/10.18653/v1/D19-5409>
- [19] Zhongyi Han, Le-Wen Cai, Wang-Zhou Dai, Yu-Xuan Huang, Benzhen Wei, Wei Wang, and Yilong Yin. 2023. Abductive subconcept learning. *Science China Information Sciences* 66, 2 (2023), 122103.
- [20] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. arXiv:2103.03874. Retrieved from <https://arxiv.org/abs/2103.03874>
- [21] Balázs Hidasi, Massimo Quadrana, Alexandros Karatzoglou, and Domonkos Tikk. 2016. Parallel recurrent neural network architectures for feature-rich session-based recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, 241–248.
- [22] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*. Retrieved from <https://openreview.net/forum?id=nZeVKeeFYf9>
- [23] Wenyue Hua, Kaijie Zhu, Lingyao Li, Lizhou Fan, Shuhang Lin, Mingyu Jin, Haochen Xue, Zelong Li, JinDong Wang, and Yongfeng Zhang. 2024. Disentangling logic: The role of context in large language model reasoning capabilities. arXiv:2406.02787. Retrieved from <https://arxiv.org/abs/2406.02787>
- [24] Mingyu Jin, Weidi Luo, Sitao Cheng, Xinyi Wang, Wenyue Hua, Ruixiang Tang, William Yang Wang, and Yongfeng Zhang. 2024. Disentangling memory and reasoning ability in large language models. arXiv:2411.13504. Retrieved from <https://arxiv.org/abs/2411.13504>
- [25] Mingyu Jin, Qinkai Yu, Dong Shu, Haiyan Zhao, Wenyue Hua, Yanda Meng, Yongfeng Zhang, and Mengnan Du. 2024. The impact of reasoning step length on large language models. In *Findings of the Association for Computational Linguistics (ACL '24)*, 1830–1842.
- [26] Woojeong Jin, Yu Cheng, Yelong Shen, Weizhu Chen, and Xiang Ren. 2022. A good prompt is worth millions of parameters: Low-resource prompt-based learning for vision-language models. In *60th Annual Meeting of the Association for Computational Linguistics*. Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.), Association for Computational Linguistics, 2763–2775. DOI : <https://doi.org/10.18653/v1/2022.acl-long.197>
- [27] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)*. IEEE, 197–206.
- [28] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2022. Decomposed prompting: A modular approach for solving complex tasks. arXiv:2210.02406. Retrieved from <https://arxiv.org/abs/2210.02406>
- [29] Saydulu Kolasani. 2023. Optimizing natural language processing, large language models (LLMs) for efficient customer service, and hyper-personalization to enable sustainable growth and revenue. *Transactions on Latest Trends in Artificial Intelligence* 4, 4 (2023).

- [30] Manling Li, Ruochen Xu Wang, Shuohang Zhou, Luowei Lin, Xudong Zhu, Chenguang Zeng, Michael Heng Ji, and Shih-Fu Chang. 2022. Clip-event: Connecting text and images with event structures. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 16420–16429.
- [31] Shuang Li, Xavier Puig, Chris Paxton, Yilun Du, Clinton Wang, Linxi Fan, Tao Chen, De-An Huang, Ekin Akyürek, Anima Anandkumar, et al. 2022. Pre-trained language models for interactive decision-making. In *36th International Conference on Neural Information Processing Systems*, 31199–31212.
- [32] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys* 55, 9 (2023), 1–35. DOI: <https://doi.org/10.1145/3560815>
- [33] Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations*. Retrieved from <https://openreview.net/forum?id=Bkg6RiCqY7>
- [34] Chen Ma, Peng Kang, and Xue Liu. 2019. Hierarchical gating networks for sequential recommendation. In *25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 825–833.
- [35] Andrea Madotto, Zhaojiang Lin, Genta Indra Winata, and Pascale Fung. 2021. Few-shot bot: Prompt-based learning for dialogue systems. arXiv:2110.08118. Retrieved from <https://arxiv.org/abs/2110.08118>
- [36] Warren S. McCulloch and Walter Pitts. 1943. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics* 5, 4 (1943), 115–133.
- [37] Xinyu Pi, Qian Liu, Bei Chen, Morteza Ziyadi, Zeqi Lin, Qiang Fu, Yan Gao, Jian-Guang Lou, and Weizhu Chen. 2022. Reasoning like program executors. In *2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 761–779. Retrieved from <https://aclanthology.org/2022.emnlp-main.48>
- [38] Meng Qu and Jian Tang. 2019. Probabilistic logic neural networks for reasoning. In *33rd International Conference on Neural Information Processing Systems*.
- [39] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI Blog* 1, 8 (2019), 9.
- [40] Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. 2021. Scaling language models: Methods, analysis & insights from training gopher. arXiv:2112.11446. Retrieved from <https://arxiv.org/abs/2112.11446>
- [41] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67.
- [42] D. Murahari Reddy, S. K. Masthan Basha, M. Chinnaiahgari Hari, and N. Penchalaiah. 2021. Dall-e: Creating images from text. *UGC Care Group I Journal* 8, 14 (2021), 71–75.
- [43] Matthew Richardson and Pedro Domingos. 2006. Markov logic networks. *Machine Learning* 62, 1–2 (2006), 107–136.
- [44] Abulhair Saparov, Richard Yuanzhe Pang, Vishakh Padmakumar, Nitish Joshi, Mehran Kazemi, Najoung Kim, and He He. 2024. Testing the general deductive reasoning capacity of large language models using OOD examples. In *37th International Conference on Neural Information Processing Systems*.
- [45] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *54th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 1715–1725. DOI: <https://doi.org/10.18653/v1/P16-1162>
- [46] Shaoyun Shi, Hanxiong Chen, Weizhi Ma, Jiaxin Mao, Min Zhang, and Yongfeng Zhang. 2020. Neural logic reasoning. In *29th ACM International Conference on Information & Knowledge Management*, 1365–1374.
- [47] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R. Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. 2022. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. arXiv:2206.04615. Retrieved from <https://arxiv.org/abs/2206.04615>
- [48] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *28th ACM International Conference on Information and Knowledge Management*, 1441–1450.
- [49] Jiayi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *11th ACM International Conference on Web Search and Data Mining*, 565–573.
- [50] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. arXiv:2302.13971. Retrieved from <https://arxiv.org/abs/2302.13971>
- [51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *31st International Conference on Neural Information Processing Systems*.
- [52] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 1–26.

- [53] Lihan Wang, Bowen Qin, Binyuan Hui, Bowen Li, Min Yang, Bailin Wang, Binhua Li, Jian Sun, Fei Huang, Luo Si, et al. 2022. Proton: Probing schema linking information from pre-trained language models for text-to-SQL parsing. In *28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22)*. ACM, New York, NY, 1889–1898. DOI: <https://doi.org/10.1145/3534678.3539305>
- [54] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. arXiv:2203.11171. Retrieved from <https://arxiv.org/abs/2203.11171>
- [55] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed. H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain of thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*. Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho (Eds.). Retrieved from https://openreview.net/forum?id=_VjQlMeSB_J
- [56] Adina Williams, Nikita Nangia, and Samuel Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In *2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 1112–1122. DOI: <https://doi.org/10.18653/v1/N18-1101>
- [57] Fen Wu, Ying Zhang, Yingce Zhang, Yiming Wu, Xingjian Chen, and Xinghua Liu. 2019. Transrec: A transformer-based recommendation model. In *25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 799–807.
- [58] Lei Xu, Yangyi Chen, Ganqu Cui, Hongcheng Gao, and Zhiyuan Liu. 2022. Exploring the universal vulnerability of prompt-based learning paradigm. In *Findings of the Association for Computational Linguistics (NAACL '22)*. Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz (Eds.), Association for Computational Linguistics, 1799–1810. DOI: <https://doi.org/10.18653/v1/2022.findings-naacl.137>
- [59] Wujiang Xu, Shaoshuai Li, Mingming Ha, Xiaobo Guo, Qiong Xu Ma, Xiaolei Liu, Linxun Chen, and Zhenfeng Zhu. 2023. Neural node matching for multi-target cross domain recommendation. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2154–2166.
- [60] Wujiang Xu, Zujie Liang, Jiaojiao Han, Xuying Ning, Wenfang Lin, Linxun Chen, Feng Wei, and Yongfeng Zhang. 2024. SLMRec: Empowering small language models for sequential recommendation. arXiv:2405.17890v1. Retrieved from <https://arxiv.org/abs/2405.17890v1>
- [61] Wujiang Xu, Qitian Wu, Runzhong Wang, Mingming Ha, Qiong Xu Ma, Linxun Chen, Bing Han, and Junchi Yan. 2024. Rethinking cross-domain sequential recommendation under open-world assumptions. In *ACM Web Conference 2024*, 3173–3184.
- [62] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2369–2380. DOI: <https://doi.org/10.18653/v1/D18-1259>
- [63] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2024. Tree of thoughts: Deliberate problem solving with large language models. In *37th International Conference on Neural Information Processing Systems*.
- [64] Kexin Yi, Jiajun Wu, Chuang Gan, Antonio Torralba, Pushmeet Kohli, and Josh Tenenbaum. 2018. Neural-symbolic VQA: Disentangling reasoning from vision and language understanding. In *Machine Learning and Logical Processing Systems 31*.
- [65] Honghua Zhang, Liunian Harold Li, Tao Meng, Kai-Wei Chang, and Guy Van den Broeck. 2022. On the paradox of learning to reason from data. arXiv:2205.11502. Retrieved from <https://arxiv.org/abs/2205.11502>
- [66] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. OPT: Open pre-trained transformer language models. arXiv:2205.01068. Retrieved from <https://arxiv.org/abs/2205.01068>
- [67] Tingting Zhang, Pengpeng Zhao, Yanchi Liu, Victor S. Sheng, Jiajie Xu, Deqing Wang, Guanfeng Liu, and Xiaofang Zhou. 2019. Feature-level deeper self-attention network for sequential recommendation. In *International Joint Conference on Artificial Intelligence*, 4320–4326.
- [68] Yuyu Zhang, Xinshi Chen, Yuan Yang, Arun Ramamurthy, Bo Li, Yuan Qi, and Le Song. 2020. Efficient probabilistic logic reasoning with graph neural networks. arXiv:2001.11850. Retrieved from <https://arxiv.org/abs/2001.11850>
- [69] Hattie Zhou, Azade Nova, Hugo Larochelle, Aaron Courville, Behnam Neyshabur, and Hanie Sedghi. 2022. Teaching algorithmic reasoning via In-context learning. arXiv:2211.09066. Retrieved from <https://arxiv.org/abs/2211.09066>
- [70] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *29th ACM International Conference on Information & Knowledge Management*, 1893–1902.

Appendix

A Logical Prompts

In the appendix, we list all the logical prompts we use in the recommendation tasks.

A.1 Not Logical Prompts

Negation

Input: For user_{user_id}: Not T is **Output:** F

Input: For user_{user_id}: Not F is **Output:** T

Double Negation

Input: For user_{user_id}: Not Not {input} is **Output:** {input}

A.2 And Logical Prompts

Identity

Input: For user_{user_id}: {input} (___) T is {input}. What operator (___) should be? **Output:** And

And

Input: For user_{user_id}: T (___) {input} is {input}. What operator (___) should be? **Output:** And

And

Annihilator

Input: For user_{user_id}: {input} (___) F is F. What operator (___) should be? **Output:** And

Input: For user_{user_id}: F (___) {input} is F. What operator (___) should be? **Output:** And

Idempotence

Input: For user_{user_id}: {input} And {input} is **Output:** {input}

Complementation

Input: For user_{user_id}: {input} (___) Not {input} is F. What operator (___) should be?

Output: And

Input: For user_{user_id}: Not {input} (___) {input} is F. What operator (___) should be?

Output: And

A.3 Or Logical Prompts

Identity

Input: For user_{user_id}: {input} (___) F is {input}. What operator (___) should be? **Output:** Or

Or

Input: For user_{user_id}: F (___) {input} is {input}. What operator (___) should be? **Output:** Or

Or

Annihilator

Input: For user_{user_id}: {input} (___) T is T. What operator (___) should be? **Output:** Or

Input: For user_{user_id}: T (___) {input} is T. What operator (___) should be? **Output:** Or

Idempotence

Input: For user_{user_id}: {input} Or {input} is **Output:** {input}

Complementation

Input: For user_{user_id}: {input} (___) Not {input} is T. What operator (___) should be?

Output: Or

Input: For user_{user_id}: Not {input} (___) {input} is T. What operator (___) should be?

Output: Or

Received 8 December 2023; revised 27 January 2025; accepted 18 March 2025